ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)

MID SEMESTER EXAMINATION **WINTER SEMESTER, 2024-2025**
DURATION: 2 HOURS **FULL MARKS: 120**

CSE 4309: Theory of Computing

Programmable calculators are not allowed. Do not write anything on the question paper.
Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

---

1. Let M be a Deterministic Finite Automaton (DFA) which can be constructed over the alphabet $\Sigma = \{0, 1\}$ as shown in Figure 3.

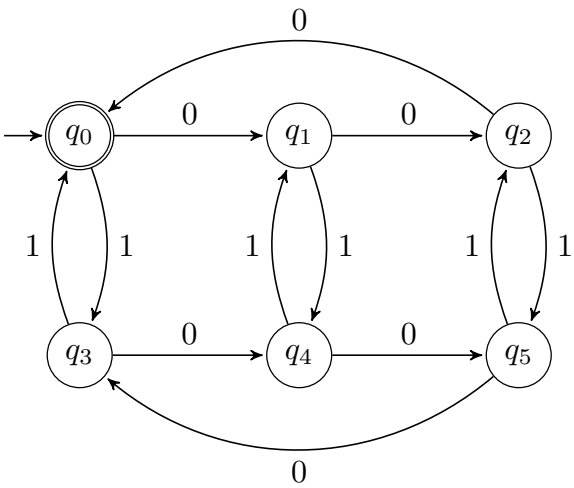

Figure 1: The DFA M for Question 1.

a) State the language recognized by M. Give an informal description in English and briefly justify your answer. 7 (CO2) (PO1)

Solution:

The language recognized by M is the set of all strings with an even number of 1's and a number of 0's that's a multiple of three. This is because horizontally, the states represent the number of 0's mod 3 and vertically, the states represent the number of 1's mod 2 (so even or odd). The top left state thus represents both an even number of 1's and a number of 0's that's a multiple of three (*i.e.*, number of 1's mod $3 = 0$).

b) Are there strings $w \in L(M)$, such that if the bits of w are flipped (changing 0 to 1 and 1 to 0), the resulting string will also be in $L(M)$? Why or why not? Is this true for every string $w \in L(M)$? (Exclude the empty string ϵ) $2 + 3 + 2$ (CO2) (PO1)

Solution:

Yes, there does exist such a string. For example, choose the string 111111000000. Since it has an even number of 1's and two multiples of 000's it is in the language $L(M)$. If the bits are flipped, it still has an even number of 1's and two multiples of 000's, so it is still in the language.
This is not true for every string, because 11000 is in $L(M)$ but when the bits are flipped, we get 00111 which is not, since it neither has an even number of 1's nor is the number of 000's.

c) Write the formal definition of $M = \langle Q, \Sigma, \delta, q_0, F \rangle$, where M is the DFA. Use a transition table to define δ. 6 (CO3) (PO1)

Solution:

The 5-tuple definition of the given DFA $M = \langle Q, \Sigma, \delta, q_{\text{start}}, F \rangle$ is as follows:

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$
$$\Sigma = \{0, 1\}$$
$$F = \{q_0\}$$
$$q_{\text{start}} = q_0$$

Transition Function δ:

State	0	1
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_0	q_5
q_3	q_4	q_0
q_4	q_5	q_1
q_5	q_3	q_2

d) i. State the regular operations. Define them with proper examples. 3 (CO1) (PO1)

Solution:

Regular operations. Let A, B be languages:

- Union: $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$
- Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\} = AB$
- Star: $A^* = \{x_1 \ldots x_k \mid \text{each } x_i \in A \text{ for } k \geq 0\}$
  Note: $\varepsilon \in A^*$ always

Example. Let $A = \{\texttt{good}, \texttt{bad}\}$ and $B = \{\texttt{boy}, \texttt{girl}\}$.

- $A \cup B = \{\texttt{good}, \texttt{bad}, \texttt{boy}, \texttt{girl}\}$
- $A \circ B = AB = \{\texttt{goodboy}, \texttt{goodgirl}, \texttt{badboy}, \texttt{badgirl}\}$
- $A^* = \{\varepsilon, \texttt{good}, \texttt{bad}, \texttt{goodgood}, \texttt{goodbad}, \texttt{badgood}, \texttt{badbad}, \texttt{goodgoodgood}, \texttt{goodgoodbad}, \ldots\}$

ii. State the different types of regular expressions. Define them with proper examples. 3 (CO1) (PO1)

Solution:

There are two types of regular expressions:

If R is atomic:

$R = a$ for $a \in \Sigma$

$R = \varepsilon$

$R = \emptyset$

If R is composite:

$$\left.\begin{array}{l} R = R_1 \cup R_2 \\ R = R_1 \circ R_2 \\ R = R_1^* \end{array}\right\} \text{Use closure constructions}$$

2. a) Construct the DFA that accepts the following regular language: 10 (CO3) (PO3)

$$L = \{w \in \{0,1\}^* \mid w \text{ is divisible by } 5 \text{ when converted to decimal}\}$$

Solution:

We process the binary string from left to right. If the current number is n, appending a bit b results in a new value $2n + b$. We track the remainder modulo 5.
Let the states represent the current remainder:

- A: remainder 0 (Start state, Accepting state)
- B: remainder 1
- C: remainder 2
- D: remainder 3
- E: remainder 4

Based on the remainder when dividing by 5:

- $5x + 0 \rightarrow$ divisible by 5 (Remainder 0) $\rightarrow$ **State A**
- $5x + 1 \rightarrow$ not divisible by 5 (Remainder 1) $\rightarrow$ **State B**
- $5x + 2 \rightarrow$ not divisible by 5 (Remainder 2) $\rightarrow$ **State C**
- $5x + 3 \rightarrow$ not divisible by 5 (Remainder 3) $\rightarrow$ **State D**
- $5x + 4 \rightarrow$ not divisible by 5 (Remainder 4) $\rightarrow$ **State E**

1. When we see input 0:
(This multiplies the current value by 2: $2 \times$ Value)

From **A**: $2(5x)$
$= 5(2x) + 0 \quad \Longrightarrow [\mathbf{A} \rightarrow \mathbf{A}]$

From **B**: $2(5x + 1)$
$= 10x + 2 = 5(2x) + 2 \quad \Longrightarrow [\mathbf{B} \rightarrow \mathbf{C}]$

From **C**: $2(5x + 2)$
$= 10x + 4 = 5(2x) + 4 \quad \Longrightarrow [\mathbf{C} \rightarrow \mathbf{E}]$

From **D**: $2(5x + 3)$
$= 10x + 6 = 5(2x + 1) + 1 \quad \Longrightarrow [\mathbf{D} \rightarrow \mathbf{B}]$

From **E**: $2(5x + 4)$
$= 10x + 8 = 5(2x + 1) + 3 \quad \Longrightarrow [\mathbf{E} \rightarrow \mathbf{D}]$

2. When we see input 1:
(Multiplies by 2 and adds 1: $2 \times \text{Value} + 1$)

From **A**: $2(5x) + 1$
$= 5(2x) + 1 \implies [\mathbf{A} \to \mathbf{B}]$

From **B**: $2(5x + 1) + 1$
$= 10x + 3 = 5(2x) + 3 \implies [\mathbf{B} \to \mathbf{D}]$

From **C**: $2(5x + 2) + 1$
$= 10x + 5 = 5(2x + 1) + 0 \implies [\mathbf{C} \to \mathbf{A}]$

From **D**: $2(5x + 3) + 1$
$= 10x + 7 = 5(2x + 1) + 2 \implies [\mathbf{D} \to \mathbf{C}]$

From **E**: $2(5x + 4) + 1$
$= 10x + 9 = 5(2x + 1) + 4 \implies [\mathbf{E} \to \mathbf{E}]$

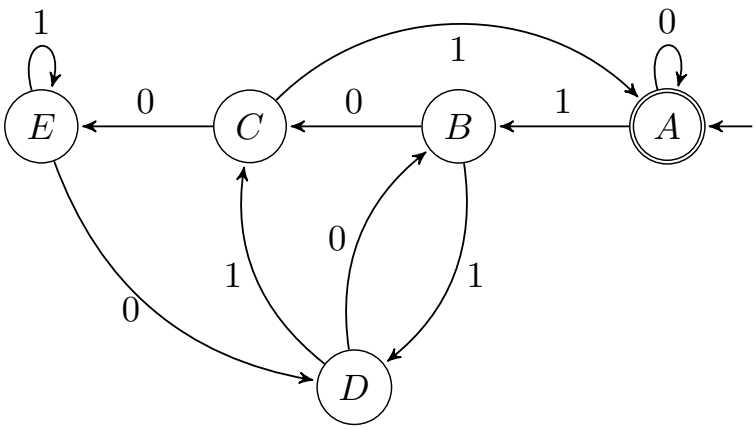

Figure 2: DFA for Question 2.a.

b) Why can we not prove that the class of regular languages is closed under the concatenation operation using DFAs? How does non-determinism help us in proving this theorem? 7 (CO2) (PO1)

Solution:

The fundamental difficulty in proving the closure of concatenation using Deterministic Finite Automata (DFAs) lies in the unknown "split point" of the input string.
Let A_1 and A_2 be two regular languages. The concatenation $L = A_1 \circ A_2$ is defined as:

$$L = \{xy \mid x \in A_1 \text{ and } y \in A_2\}$$

For a machine to accept a string $w \in L$, it must be able to partition w into two substrings, x and y. The fundamental problem is that the location of the "split point" between x and y is unknown.
To prove this, the idea is to take two NFAs, N_1 and N_2 for A_1 and A_2, and combine them into a new NFA N as we did for the case of union, but this time in a different way, as shown in Figure 3.
Assign N's start state to be the start state of N_1. The accept states of N_1 have additional ε arrows that nondeterministically allow branching to N_2 whenever N_1 is in an accept state, signifying that it has found an initial piece of the input that constitutes a string in A_1. The

accept states of N are the accept states of N_2 only. Therefore, it accepts when the input can be split into two parts, the first accepted by N_1 and the second by N_2. We can think of N as nondeterministically guessing where to make the split.

Proof. Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1, and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2. Construct $N = (Q, \Sigma, \delta, q_1, F_2)$ to recognize $A_1 \circ A_2$.

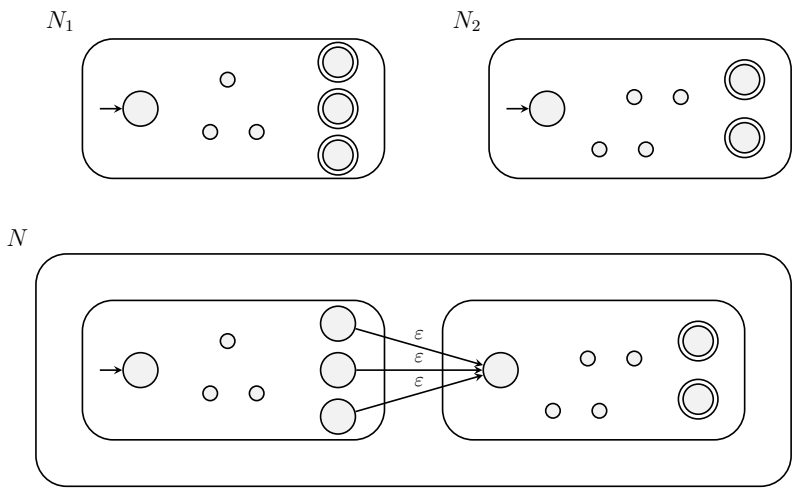

Figure 3: Construction of N to recognize $A_1 \circ A_2$

1. $Q = Q_1 \cup Q_2$.

   The states of N are all the states of N_1 and N_2.

2. The state q_1 is the same as the start state of N_1.

3. The accept states F_2 are the same as the accept states of N_2.

4. Define δ so that for any $q \in Q$ and any $a \in \Sigma_\varepsilon$,

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ and } q \notin F_1, \\ \delta_1(q,a) & q \in F_1 \text{ and } a \neq \varepsilon, \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1 \text{ and } a = \varepsilon, \\ \delta_2(q,a) & q \in Q_2. \end{cases}$$

□

c) Construct DFAs for the following languages: 2×4 (CO3) (PO3)

i. $L = \{w \in \{0,1\}^* \mid$ the symbol in every odd position of w is $1\}$ [Assume 1-based indexing]

Solution:

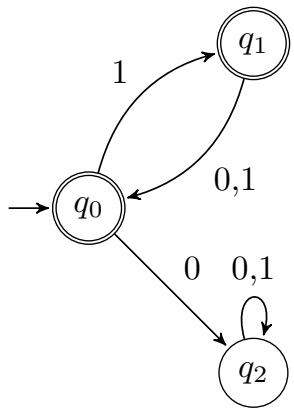

Figure 4: DFA for Question 2.c)i.

ii. $L = \{w \in \{\mathtt{a},\mathtt{b}\}^* \mid w$ has at least two $\mathtt{a}$'s and at most one $\mathtt{b}\}$

Solution:

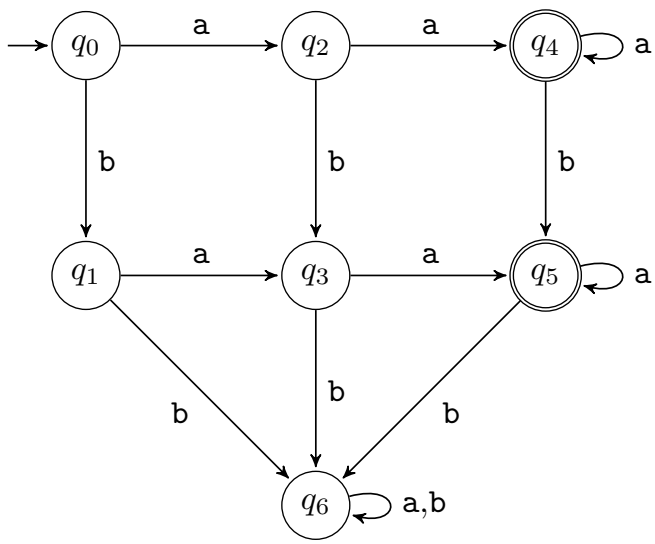

Figure 5: DFA for Question 2.c)ii.

d) To turn on a computer, you need to press the power button of the computer. Whether you long-press or short-press the button, the result is the same. When the computer is on, if you long-press the power button, it shuts down and if you short-press the button, it restarts. When the computer is restarting, it continues to restart even if you press the power button. Draw a DFA to design the aforementioned scenario. 5 (CO3) (PO2)

Solution:

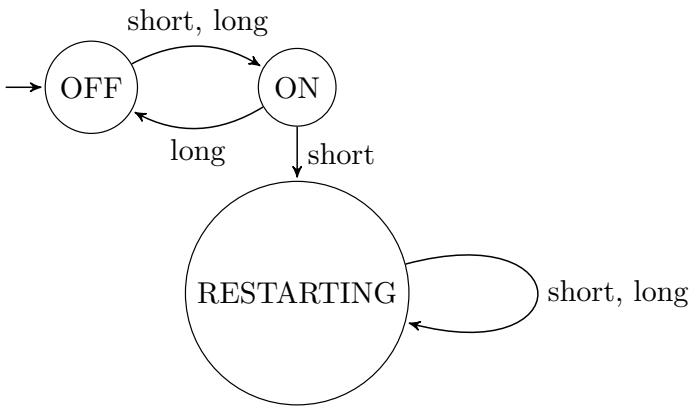

Figure 6: DFA for Question 2.d.

3. a) Write regular expressions that represent each of the following languages: 3×6 (CO3) (PO3)

i. $L = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid w\text{'s 9th symbol from the left is } \mathtt{b}\}$

Solution:

$$R = (\mathtt{a} + \mathtt{b})^8 \mathtt{b} (\mathtt{a} + \mathtt{b})^*$$

ii. $L = \{w \in \mathtt{a}^n \mathtt{b}^m \mid n + 2m \equiv 0 \pmod 3\}$

Solution:

The condition $n + 2m \equiv 0 \pmod 3$ can be broken down into cases based on the values of $n \pmod 3$ and $m \pmod 3$: If $m \pmod 3 \equiv 0$, then $n \pmod 3 \equiv 0$. If $m \pmod 3 \equiv 1$, then $n \pmod 3 \equiv 1$. If $m \pmod 3 \equiv 2$, then $n \pmod 3 \equiv 2$.
Thus n and m must have the same remainder modulo 3. There are three possible cases:

$$n \equiv m \equiv 0,\ 1,\ \text{or } 2 \pmod 3.$$

For each case, the corresponding regular expression over strings of the form $\mathtt{a}^n \mathtt{b}^m$ is:

$$(\mathtt{a}^3)^*(\mathtt{b}^3)^*, \quad (\mathtt{a}^3)^*\mathtt{a}(\mathtt{b}^3)^*\mathtt{b}, \quad (\mathtt{a}^3)^*\mathtt{aa}(\mathtt{b}^3)^*\mathtt{bb}.$$

Taking their union (denoted by $+$), a regular expression for R is:

$$R = (\mathtt{a}^3)^*(\mathtt{b}^3)^* + (\mathtt{a}^3)^*\mathtt{a}(\mathtt{b}^3)^*\mathtt{b} + (\mathtt{a}^3)^*\mathtt{aa}(\mathtt{b}^3)^*\mathtt{bb}$$

iii. $L \subseteq \{\mathtt{a}, \mathtt{b}\}^*$ is the language of the DFA shown in Figure 7.

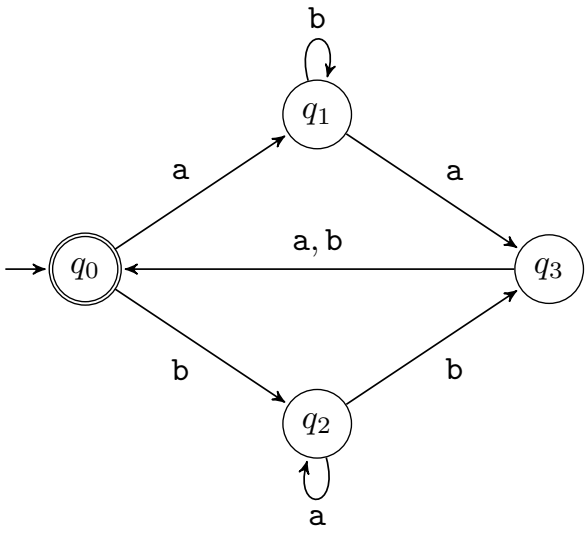

Figure 7: The DFA for Question 3.a)iii.

Solution:

$$\left(\mathtt{ab^*a(a+b)} \;+\; \mathtt{ba^*b(a+b)}\right)^*$$

b) Prove Arden's theorem, which is stated as follows: 7
If P and Q are two regular expressions over Σ, and if P does not contain ϵ, then the following equation in R given by $R = Q + RP$, has a unique solution $R = QP^*$. (CO2) (PO1)

Solution:

Proof. We prove the result in two steps.

(1) Existence: QP^* is a solution.
Substitute $R = QP^*$ into the right-hand side of the equation:

$$Q + RP = Q + QP^*P.$$

Since $P^*P = P^+$ and $\varepsilon + P^+ = P^*$, we obtain

$$Q + QP^*P = Q(\varepsilon + P^+) = QP^*.$$

Hence,

$$R = QP^*$$

satisfies the equation $R = Q + RP$.

(2) Uniqueness: QP^* is the only solution.
Let R be any solution of $R = Q + RP$. By repeated substitution, we have

$$\begin{aligned} R &= Q + RP \\ &= Q + (Q + RP)P \\ &= Q + QP + RP^2 \\ &= Q + QP + QP^2 + \cdots + QP^n + RP^{n+1}. \end{aligned}$$

Since $\varepsilon \notin L(P)$, no word in P^{n+1} has length less than $n+1$. Letting $n \to \infty$, the term RP^{n+1} contributes no finite words, and thus

$$R = Q + QP + QP^2 + QP^3 + \cdots = QP^*.$$

Therefore, any solution R must equal QP^*.

Hence, QP^* is the unique solution of $R = Q + RP$. □

c) Draw NFAs for the following regular expressions ($\Sigma = \{\mathtt{a}, \mathtt{b}\}$): 2×7 (CO3) (PO3)

i. $(\mathtt{a} \cup \mathtt{b}^+)\mathtt{a}^+\mathtt{b}^+$

Solution:

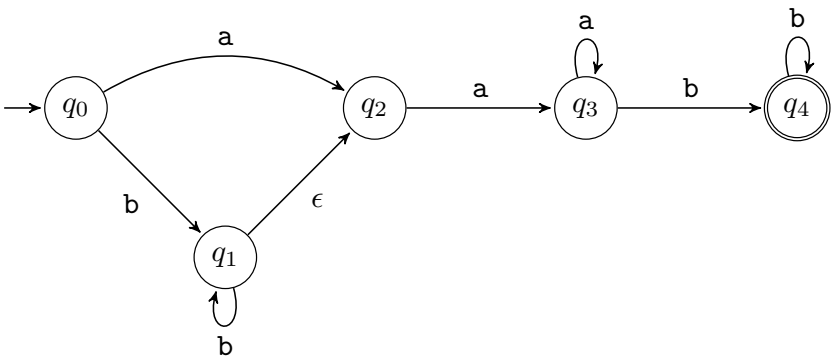

Figure 8: DFA for Question 3.c)i.

ii. $\mathtt{a}^+ \cup (\mathtt{ab})^+$

Solution:

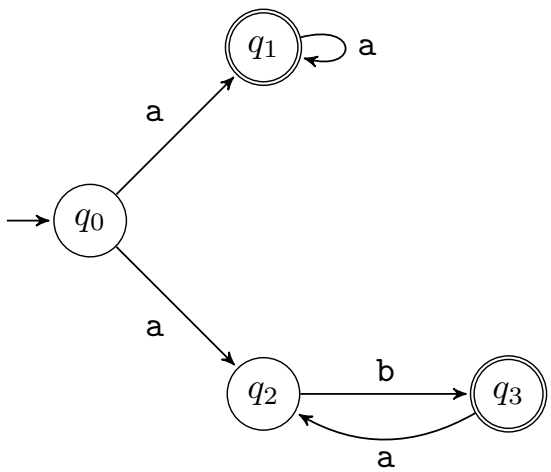

Figure 9: DFA for Question 3.c)ii.

4. a) State the pumping lemma. 5 (CO1) (PO1)

Solution:

Pumping Lemma: For every regular language A, there is a number p (the "pumping length") such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

1. $xy^iz \in A$ for all $i \geq 0$ $\quad y^i = \underbrace{yy \cdots y}_{i}$
2. $y \neq \varepsilon$
3. $|xy| \leq p$

b) Using the pumping lemma, prove the following: 2×10

i. $L = \{\mathtt{a}^{2^n} \mid n \geq 0\}$ is not a regular language. (CO2) (PO1)

Solution:

Consider the language, $L = \{a^{2^n} \mid n \geq 0\}$ (Here, a^{2^n} means a string of 2^n a's).

Assume that L is regular language.

Let p be the pumping length given by pumping lemma consider a string $S = a^{2^p} \in L$. And $|S| > p$

By pumping lemma, this string can be divided into three pieces xyz such that $|xy| \leq p, |y| > 0$ and $xy^iz \in L, \forall i \geq 0$

Let $aaaa$ be the string that belongs to L. The pumping length of the string is 2. To satisfy the conditions of the pumping lemma, $x = a, y = a, z = aa$.

$$\begin{aligned} S &= aaaa \\ &= \underset{x}{a}\,\underset{y}{a}\,\underset{z}{aa} \end{aligned}$$

Pump the middle part such that xy^iz $(i \geq 0)$. For $i = 2$, the y becomes aa. The string after pumping is $aaaaa$.

$$\begin{aligned} S &= (a)(a)^i(aa) \\ &= \underset{x}{a}\,\underset{y}{aa}\,\underset{z}{aa} \quad [\text{when } i = 2] \end{aligned}$$

The string $aaaaa \notin L$. It is a contradiction. So, the pumping lemma is violated.

Therefore, L is not a regular language.

ii. $L = \{1^p \mid p \text{ is a prime number}\}$ is not a regular language.

Solution:

Consider the language, $L = \{1^p \mid p \text{ is a prime number}\}$.

Assume that L is a regular language.

Let k be the pumping length given by the pumping lemma. Consider a string $S = 1^q \in L$, where q is a prime number and $q \geq k$.

By pumping lemma, this string can be divided into three pieces xyz such that $|xy| \leq k, |y| > 0$ and $xy^iz \in L, \forall i \geq 0$. Let $|y| = m$.

Since $|y| > 0$, we know $m \geq 1$.

$$\begin{aligned} S &= 1^q \\ &= \underset{x}{1\ldots1}\,\underset{y}{1\ldots1}\,\underset{z}{1\ldots1} \quad [\text{where } |y| = m] \end{aligned}$$

Pump the middle part y exactly $q + 1$ times (set $i = q + 1$).

$$\begin{aligned} S' &= xy^{q+1}z \\ &= xyy^qz \end{aligned}$$

The length of the new string $|S'|$ is calculated as follows:

$$\begin{aligned} |S'| &= |xyz| + |y^q| \\ &= q + q|y| \\ &= q + qm \\ &= q(1+m) \end{aligned}$$

Since $q \geq 2$ (it is prime) and $m \geq 1$, then $(1+m) \geq 2$. Thus, the length $q(1+m)$ is a composite number (it has factors other than 1 and itself).

The string $S' \notin L$ because its length is not prime. It is a contradiction. So, the pumping lemma is violated.

Therefore, L is not a regular language.

MARKS SUMMARY TABLE

Question	Marks	Keywords
1.a	7	
1.b	7	
1.c	6	
1.d)i	3	
1.d)ii	3	
2.a	10	
2.b	7	
2.c	8	
2.d	5	
3.a	18	
3.b	7	
3.c	14	
4.a	5	
4.b	20	
Total	120	